

LAB # 7b

Implementation of 2D arrays in java

Objective

To gain familiarity with the Implementation of 2D arrays in java.

Lab Goals

To train on the construction of arrays in java and show how java treats arrays and the various elements of an arrays.

Introduction to 2D Arrays in Java

A 2D array in Java is an array of arrays. It's essentially a table where each element is an array itself. This structure is used to store data in rows and columns format.

Syntax:

```
dataType[][] arrayName;
```

Code:

```
int[][] matrix;
```

Defining 2D Arrays:

Defining a 2D array involves specifying its size and optionally initializing it.

Syntax:

```
dataType[][] arrayName = new dataType[rows][columns];
```

Code:

```
int[][] matrix = new int[3][3];
```

Declaring 2D Array Variables:

You can declare a 2D array variable without allocating memory immediately.

Syntax:

```
dataType[][] arrayName;
```

Example Code:

```
public class Declare2DArray {  
    public static void main(String[] args) {  
        int[][] matrix;  
        System.out.println("2D array variable declared.");  
    }  
}
```

```
}  
}
```

Output:

```
2D array variable declared.
```

Initializing 2D Arrays:

You can initialize a 2D array at the time of declaration or later in the code.

Syntax:

```
dataType[][] arrayName = {  
    {value1, value2},  
    {value3, value4}  
};
```

Example Code:

```
public class Initialize2DArray {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
  
        // Printing the 2D array  
        for (int i = 0; i < matrix.length; i++) {  
            for (int j = 0; j < matrix[i].length; j++) {  
                System.out.print(matrix[i][j] + " ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Output:

```
1 2 3  
4 5 6  
7 8 9
```

Accessing 2D Arrays:

You can access elements of a 2D array using row and column indices.

Syntax:

```
arrayName[row][column]
```

Example Code:

```
public class Access2DArray {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Accessing and printing a specific element
        int element = matrix[1][2]; // Accessing element at row 1, column 2
        System.out.println("Element at row 1, column 2: " + element);
    }
}
```

Output:

```
Element at row 1, column 2: 6
```

Reusing 2D Array Variables:

You can reuse a 2D array variable by reassigning it to a new 2D array.

Syntax:

```
arrayName = new dataType[rows][columns];
```

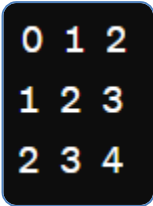
Example Code:

```
public class Reuse2DArray {
    public static void main(String[] args) {
        int[][] matrix = new int[2][2];
        matrix[0][0] = 1;
        matrix[0][1] = 2;
        matrix[1][0] = 3;
        matrix[1][1] = 4;

        // Reusing the 2D array variable
        matrix = new int[3][3];
    }
}
```

```
// Initialize the new array
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        matrix[i][j] = i + j;
    }
}

// Printing the reused and reinitialized 2D array
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
}
```

Output:

0	1	2
1	2	3
2	3	4

2D Array Length:

You can get the length of a 2D array (number of rows) and the length of a specific row (number of columns).

Syntax:

```
arrayName.length // Number of rows
arrayName[row].length // Number of columns in a specific row
```

Example Code:

```
public class ArrayLength2D {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Getting the number of rows
```

```
int rows = matrix.length;
// Getting the number of columns in the first row
int cols = matrix[0].length;

System.out.println("Number of rows: " + rows);
System.out.println("Number of columns in first row: " + cols);
}
}
```

Output:

```
Number of rows: 3
Number of columns in first row: 3
```

Exercise

- A. Write a program to create a rectangular array containing a multiplication table from 1 x 1 up to 12 x 12. Output the table as 13 columns with the numeric values right aligned in columns. (The first line of the output will be the column headings, the first column with no heading, then the numbers 1 to 12 for the remaining columns. The first item in each of the succeeding lines is the row heading, which ranges from 1 to 12.).
- B. Write a program to find the transpose of a 2x3 matrix.
- C. Write a program to check if a 3x3 2D array is symmetric or not. A 2D array is symmetric **if** **matrix[i][j] == matrix[j][i]** for all valid **i** and **j**.
- D. Write a program to find the sum of each row in a 3x3 2D array and print the sum for each row.
- E. Write a program to print a pyramid pattern of asterisks with 5 rows.